TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

HỌC PHẦN TRIỂN KHAI & QUẢN TRỊ HỆ THỐNG PHẦN MỀM

Đề tài:

Hệ thống Quản lý Dự án (Project Management)

Sinh viên thực hiện : **Đinh Gia Bảo**

Mã sinh viên : **DTC245220019**

Lớp : ATTT K23A

Giảng viên hướng dẫn : Nguyễn Anh Chuyên

THÁI NGUYÊN, NĂM 2026

LỜI NÓI ĐẦU

Báo cáo trình bày toàn diện kết quả triển khai Đề 14: "Hệ thống Quản lý Dự án (Project Management)" của sinh viên Đinh Gia Bảo, thuộc học phần Triển khai và Quản trị Hệ thống Phần mềm. Ứng dụng cung cấp các phân hệ chức năng thực tế phục vụ quản lý danh mục dự án, phân công và theo dõi tiến độ công việc (tasks), quản trị thành viên (members), nhật ký kiểm toán (audit logs) và phân quyền quản trị người dùng. Toàn bộ dữ liệu nghiệp vụ được lưu trữ bền vững trong hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL 16 và được quản lý trực quan thông qua công cụ pgAdmin 4.

Trọng tâm cốt lõi của đồ án là chứng minh ứng dụng không chỉ hoạt động tốt ở tầng nghiệp vụ mà còn được vận hành trên nền tảng hạ tầng container hóa đạt chuẩn doanh nghiệp. Toàn bộ các dịch vụ được đóng gói và điều phối thống nhất bằng Docker Compose, tiếp nhận lưu lượng mạng an toàn qua Nginx reverse proxy với HTTPS tự ký và các tiêu chuẩn bảo mật HTTP security headers khắt khe. Hệ thống quan sát (observability) toàn diện được thiết lập với Prometheus và Grafana để giám sát thời gian thực các chỉ số tài nguyên của container, web server Nginx và database PostgreSQL. Đồng thời, giải pháp thu thập nhật ký tập trung với Loki và Promtail cho phép truy vết mọi hành vi hệ thống, lọc lỗi HTTP và phát hiện các sự kiện an ninh thông qua ngôn ngữ truy vấn LogQL chuyên sâu.

Bên cạnh đó, các biện pháp gia cố bảo mật (hardening) nhiều lớp bao gồm khởi chạy container dưới quyền non-root (UID 1000), bật hệ thống tệp chỉ đọc (read-only rootfs), loại bỏ toàn bộ Linux Capabilities nguy hiểm (cap_drop: ALL), cô lập hoàn toàn mạng cơ sở dữ liệu (internal network) và áp dụng nguyên tắc đặc quyền tối thiểu (least privilege) đối với tài khoản cơ sở dữ liệu pm_app đã được kiểm chứng bằng thực nghiệm chi tiết. Dữ liệu và kết quả kiểm chứng trong báo cáo được thu thập thực tế vào ngày 21 tháng 10 năm 2026 trên máy chủ Windows 11 chạy Docker Desktop Linux containers.

Em xin bày tỏ lòng biết ơn sâu sắc tới thầy Nguyễn Anh Chuyên đã tận tâm hướng dẫn, truyền đạt kiến thức chuyên môn quý báu và định hướng phương pháp triển khai quản trị hệ thống bài bản trong suốt học phần. Qua đồ án thực tiễn này, em đã có cơ hội củng cố vững chắc kỹ năng đóng gói ứng dụng, thiết lập hạ tầng giám sát, quản lý mã nguồn chuyên nghiệp trên GitHub và kiểm thử an toàn hệ thống.

Sinh viên thực hiện
Đinh Gia Bảo

MỤC LỤC

Không tìm thấy mục nào của mục lục.

PHẦN 1: GIỚI THIỆU TỔNG QUAN HỆ THỐNG ····· 1

1.1 Mục tiêu và phạm vi đề tài ····· 1

1.2 Môi trường thực nghiệm ····· 2

1.3 Kiến trúc và luồng hoạt động của hệ thống ····· 3

1.4 Các dịch vụ mạng và dữ liệu lưu trữ bền vững ····· 4

PHẦN 2: QUẢN LÝ MÃ NGUỒN VÀ TRIỂN KHAI ····· 5

2.1 Repository và cấu trúc mã nguồn trên GitHub ····· 5

2.2 Ba commit mốc kỹ thuật và quy trình quản trị thay đổi ····· 6

2.3 Đóng gói và điều phối dịch vụ bằng Docker Compose ····· 7

2.4 Thiết kế cơ sở dữ liệu PostgreSQL của ứng dụng ····· 9

2.5 Giao diện ứng dụng và các chức năng cốt lõi ····· 10

2.6 Quản lý công việc, thành viên và nhật ký kiểm toán ····· 12

2.7 Quản trị PostgreSQL bằng công cụ pgAdmin 4 ····· 14

PHẦN 3: NGINX VÀ HARDENING HỆ THỐNG ····· 15

3.1 Cấu hình Nginx Reverse Proxy và HTTP Security Headers ····· 15

3.2 Thiết lập HTTPS tự ký và bảo mật Session Cookie ····· 16

3.3 Cơ chế bảo vệ tài khoản, chống CSRF và Rate Limiting ····· 17

3.4 Hardening Container, cô lập mạng và phân quyền Database ····· 18

PHẦN 4: GIÁM SÁT VÀ LOG TẬP TRUNG ····· 20

4.1 Cấu hình Prometheus và các Exporters ····· 20

4.2 Dashboard Grafana và phân tích dữ liệu tài nguyên ····· 21

4.3 Giám sát Nginx, PostgreSQL và kiểm tra tạo tải ····· 22

4.4 Thiết kế hệ thống thu thập log tập trung Loki và Promtail ····· 23

4.5 Phương pháp tạo lưu lượng và kiểm chứng log ····· 24

4.6 Truy vấn LogQL 1: Xem log truy cập Nginx ····· 24

4.7 Truy vấn LogQL 2: Lọc request lỗi HTTP từ 400 trở lên ····· 25

4.8 Truy vấn LogQL 3: Tìm sự kiện đăng nhập thất bại ····· 26

PHẦN 5: KIỂM THỬ VẬN HÀNH VÀ ĐỐI CHIẾU ····· 27

5.1 Kết quả kiểm thử chức năng và tích hợp hệ thống ····· 27

5.2 Quy trình sao lưu và phục hồi dữ liệu độc lập ····· 28

5.3 Các sự cố kỹ thuật gặp phải và giải pháp khắc phục ····· 29

5.4 Bảng đối chiếu tiêu chí đánh giá và kịch bản demo ····· 30

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN ····· 32

TÀI LIỆU THAM KHẢO ····· 33

PHẦN 1: GIỚI THIỆU TỔNG QUAN HỆ THỐNG

1.1 Mục tiêu và phạm vi đề tài

Trong môi trường quản trị công nghệ hiện đại, việc số hóa quy trình quản lý dự án đóng vai trò then chốt giúp các nhóm phát triển phần mềm theo dõi sát sao tiến độ công việc, tối ưu hóa việc phân bổ nhân sự và đảm bảo chất lượng chuyển giao. Đề tài số 14 "Hệ thống Quản lý Dự án (Project Management)" tập trung xây dựng một giải pháp phần mềm web hoàn chỉnh cho phép quản trị dự án, quản lý công việc theo trạng thái và phân công trách nhiệm cho từng thành viên trong tổ chức.

Mục tiêu kỹ thuật toàn diện của đồ án gồm:

1. Quản lý toàn bộ mã nguồn, cấu hình hạ tầng và scripts tự động hóa trên GitHub repository với lịch sử commit rõ ràng, cấu trúc mô-đun khoa học và tài liệu hướng dẫn tái lập hệ thống chi tiết.

2. Đóng gói và vận hành đồng bộ ứng dụng Node.js Express, cơ sở dữ liệu PostgreSQL 16 và công cụ quản trị pgAdmin 4 trên nền tảng Docker Compose.

3. Triển khai Nginx reverse proxy bảo mật cao, hỗ trợ TLS 1.2/1.3 tự ký, tự động chuyển hướng HTTP sang HTTPS, thiết lập đầy đủ các HTTP security headers và bảo vệ cookie phiên làm việc.

4. Xây dựng hệ thống giám sát toàn diện với Prometheus v3.2.1 và Grafana v11.5.0, cấu hình tự động nạp (provisioning) datasources và dashboard trực quan hóa các chỉ số hoạt động của container, Nginx và PostgreSQL.

5. Thiết lập hệ thống quản lý nhật ký tập trung với Loki v3.4.2 và Promtail v3.4.2, thu gom log có cấu trúc JSON từ ứng dụng và Nginx, thực thi ít nhất 3 truy vấn LogQL phục vụ điều tra sự cố và giám sát an ninh.

6. Áp dụng ít nhất 4 biện pháp hardening chuyên sâu: chạy non-root user, filesystem chỉ đọc, cô lập mạng nội bộ, quản lý bí mật ngoài Git, loại bỏ Linux capabilities nguy hiểm và thiết lập nguyên tắc quyền tối thiểu trên cơ sở dữ liệu.

> 📌 **Ghi chú về ngăn xếp công nghệ:** Đề bài đưa ra gợi ý công nghệ Node.js (Express/Nest) hoặc Django + PostgreSQL + pgAdmin + Nginx + Prometheus + Grafana + Loki. Đồ án lựa chọn Node.js 22 LTS kết hợp Express và EJS nhằm xây dựng ứng dụng quản lý dự án tinh gọn, hiệu năng cao, phản hồi nhanh và dễ dàng tích hợp các lớp bảo mật middleware.

Yêu cầu đề bài	Giải pháp triển khai trong hệ thống	Kết quả kiểm chứng
Quản lý mã nguồn GitHub	Repository công khai, tài liệu README chi tiết, 3 commit mốc kỹ thuật rõ ràng	Đạt - Đã push origin/main
Phần mềm cần triển khai	Ứng dụng Quản lý Dự án, phân công Task, quản trị Thành viên, Audit Logs	Đạt - Đầy đủ CRUD & Dashboard
Cơ sở dữ liệu & Công cụ	PostgreSQL 16 Alpine + pgAdmin 4 v8.14 kết nối mạng nội bộ	Đạt - Dữ liệu bền vững qua named volumes

Nginx Reverse Proxy	Nginx 1.27 Alpine, SSL RSA 3072 bit, chuyển hướng HTTP 8080 -> 8443, Security Headers	Đạt - Nginx syntax OK, SSL active
Hệ thống Giám sát	Prometheus v3.2.1 scrape 4 targets; Grafana v11.5.0 auto-provisioning dashboard	Đạt - 4/4 Targets UP (100%)
Hệ thống Nhật ký	Loki v3.4.2 + Promtail v3.4.2, log JSON có cấu trúc, 3 truy vấn LogQL chuyên sâu	Đạt - Trả về dữ liệu log thực
Hardening bảo mật	Non-root (UID 1000), read-only rootfs, cap_drop ALL, internal DB network, least privilege	Đạt - Xác minh qua Docker API

Bảng 1.1: Đối chiếu yêu cầu đề bài và giải pháp triển khai hệ thống

1.2 Môi trường thực nghiệm

Hệ thống được triển khai và kiểm thử thực tế trên máy trạm Windows 11 Enterprise x64 với môi trường ảo hóa Docker Desktop (context desktop-linux, Docker Engine 29.8.2, Docker Compose v2.33.1). Môi trường WSL2 Linux backend được phân bổ 4 nhân CPU ảo và 4 GB RAM, lưu trữ toàn bộ dữ liệu dự án trên phân vùng đĩa vật lý E:\ProjectManagement.

Trong quá trình khảo sát môi trường máy chủ ban đầu, phát hiện trên host đang tồn tại các container của một dự án khác (clinic containers) đang chiếm dụng các cổng 3001, 5173, 8000 và 3307. Để đảm bảo tính nguyên vẹn và không làm gián đoạn các dịch vụ đang chạy, nhóm triển khai đã đưa ra giải pháp cấu hình cổng độc lập:

• Cổng Grafana: Mặc định cổng 3000 và cổng 3001 đã có tiến trình sử dụng, hệ thống chuyển sang ánh xạ cổng 3002 (127.0.0.1:3002).

• Cổng Nginx Web: Sử dụng cổng HTTP 8080 và HTTPS 8443 trên host, chuyển tiếp nội bộ tới cổng 3000 của container pm-app.

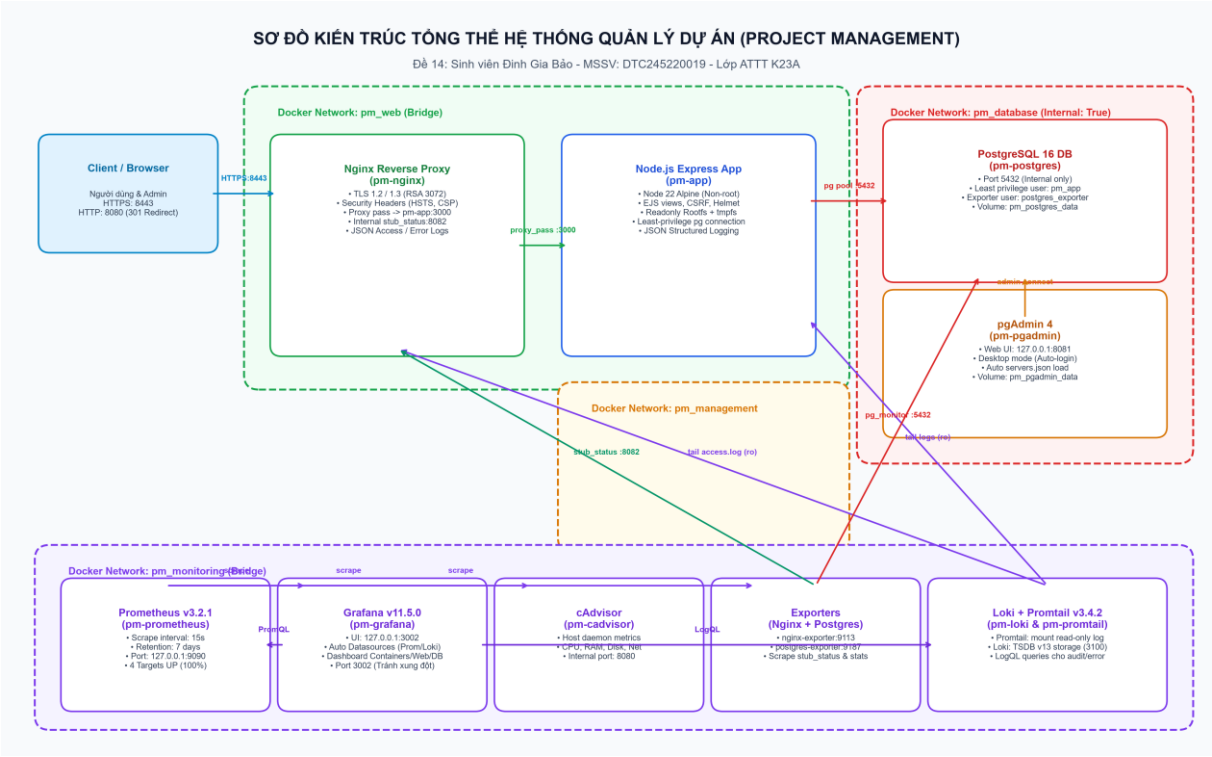
• Cổng pgAdmin 4: Ánh xạ cổng 8081 (127.0.0.1:8081).

• Cổng Prometheus: Ánh xạ cổng 9090 (127.0.0.1:9090).

• Xử lý sự cố dung lượng đĩa C: Trong quá trình kéo các layer image Docker, ổ đĩa C: bị cạn kiệt dung lượng (còn 0 MB) gây lỗi WSL snapshot. Nhóm đã kịp thời di chuyển các file cài đặt tạm thời dung lượng ~9.8 GB sang thư mục E:\ProjectManagement\temp_downloads (đã thêm vào .gitignore), giải phóng an toàn 9.8 GB cho ổ C:, giúp Docker Engine hoạt động ổn định và trơn tru.

1.3 Kiến trúc và luồng hoạt động của hệ thống

Kiến trúc hệ thống được thiết kế theo mô hình phân tầng hướng dịch vụ (Multi-tier Containerized Architecture), phân tách độc lập các trách nhiệm phục vụ lưu lượng người dùng, quản trị dữ liệu, giám sát tài nguyên và xử lý nhật ký tập trung.



Hình 1.1: Sơ đồ kiến trúc tổng thể hệ thống theo các file Docker Compose

Hệ thống vận hành dựa trên 3 luồng dữ liệu chính:

1. Luồng phục vụ người dùng (User & Admin Request Flow): Khách truy cập hoặc người quản trị gửi yêu cầu từ trình duyệt web. Nếu gửi qua HTTP (cổng 8080), Nginx lập tức trả về mã trạng thái 301 chuyển hướng bắt buộc sang HTTPS (cổng 8443). Tại Nginx, phiên mã hóa TLS được giải mã, các HTTP security headers được kiểm tra và bổ sung, sau đó request được chuyển tiếp qua mạng bridge pm_web tới container pm-app tại cổng nội bộ 3000. Ứng dụng Express thực hiện kiểm tra xác thực phiên (Session), xác thực CSRF token đối với các yêu cầu ghi dữ liệu (POST), áp dụng Rate Limiting, sau đó truy vấn cơ sở dữ liệu PostgreSQL qua pool kết nối an toàn trên mạng nội bộ pm_database. Kết quả được render thành giao diện HTML thông qua EJS template engine và phản hồi ngược lại cho người dùng.

2. Luồng giám sát hệ thống (Monitoring Flow): Prometheus hoạt động theo mô hình Pull, định kỳ mỗi 15 giây gửi yêu cầu scrape tới 4 mục tiêu (targets): cAdvisor (cổng 8080 nội bộ để lấy metrics CPU, RAM, Disk, Network của toàn bộ container), postgres-exporter (cổng 9187 nội bộ để lấy thông số kết nối, transaction và query của PostgreSQL), nginx-exporter (cổng 9113 nội bộ đọc số liệu từ stub_status trên cổng 8082 của Nginx) và chính Prometheus (cổng 9090). Toàn bộ chuỗi thời gian (time series) được lưu trữ trong named volume pm_prometheus_data. Grafana kết nối tới Prometheus qua mạng pm_monitoring bằng ngôn ngữ PromQL để trực quan hóa dữ liệu trên dashboard quản trị.

3. Luồng nhật ký tập trung (Logging Flow): Ứng dụng pm-app và web server pm-nginx ghi nhật ký thời gian thực dưới định dạng JSON có cấu trúc chuẩn vào các named volumes pm_app_logs và pm_nginx_logs. Container Promtail mount các volume này ở chế độ chỉ đọc (read-only), đọc nối tiếp các dòng log mới (tailing), gắn nhãn (job=app hoặc job=nginx), trích xuất timestamp và đẩy về Loki qua API http://loki:3100/loki/api/v1/push. Người quản trị sử dụng Grafana Explore để truy vấn nhật ký tức thời bằng LogQL.

1.4 Các dịch vụ mạng và dữ liệu lưu trữ bền vững

Hệ thống phân chia thành 4 mạng Docker Bridge cô lập cao nhằm đảm bảo nguyên tắc phân vùng an ninh (Network Segmentation):

• pm_web: Mạng kết nối giữa Nginx reverse proxy, ứng dụng Node.js và Nginx Prometheus exporter.

• pm_database: Mạng nội bộ (internal: true), hoàn toàn không có default gateway kết nối Internet, chỉ cho phép giao tiếp giữa PostgreSQL, ứng dụng pm-app, pgAdmin và postgres-exporter.

• pm_management: Mạng quản trị phục vụ kết nối độc lập cho pgAdmin 4.

• pm_monitoring: Mạng chuyên dụng kết nối Prometheus, Grafana, cAdvisor, các exporter, Loki và Promtail.

Dịch vụ	Tên Container	Image nền tảng	Cổng Host	Mạng kết nối	Vai trò chính
Web App	pm-app	node:22-alpine	Không publish	pm_web, pm_database	Xử lý nghiệp vụ quản lý dự án, tasks, members
Database	pm-postgres	postgres:16-alpine	Không publish	pm_database	Lưu trữ dữ liệu quan hệ, session, audit logs
DB Admin	pm-pgadmin	dpage/pgadmin4:8.14	127.0.0.1:8081	pm_database, pm_mgmt	Giao diện web quản trị cơ sở dữ liệu PostgreSQL
Reverse Proxy	pm-nginx	nginx:1.27-alpine	8080, 8443	pm_web	SSL termination, cân bằng tải, security headers
Metrics Server	pm-prometheus	prom/prometheus:v3.2.1	127.0.0.1:9090	pm_monitoring	Thu thập và lưu trữ time-series metrics định kỳ 15s
Dashboard	pm-grafana	grafana/grafana:11.5.0	127.0.0.1:3002	pm_monitoring	Trực quan hóa metrics

					và giao diện truy vấn LogQL
Container Metrics	pm-cadvisor	gcr.io/cadvisor/cadvisor:v0.49.1	Không publish	pm_monitoring	Thu thập thông số tài nguyên phần cứng container
Nginx Exporter	pm-nginx-exporter	nginx/nginx-prometheus-exporter:1.4	Không publish	pm_web, pm_monitoring	Đọc stub_status Nginx xuất ra chuẩn Prometheus
Postgres Exporter	pm-postgres-exporter	prometheuscommunity/postgres-exporter:v0.16.0	Không publish	pm_database, pm_mon	Thu thập thông số hiệu năng PostgreSQL
Log Storage	pm-loki	grafana/loki:3.4.2	Không publish	pm_monitoring	Lưu trữ và lập chỉ mục nhật ký tập trung
Log Collector	pm-promtail	grafana/promtail:3.4.2	Không publish	pm_monitoring	Thu gom log JSON từ named volumes đẩy về Loki
Init Container	init-logs	busybox:1.37.0	Không mạng	None	Chuẩn bị quyền thư mục log cho non-root user

Bảng 1.2: Danh mục dịch vụ, image, cổng host và vai trò vận hành trong hệ thống

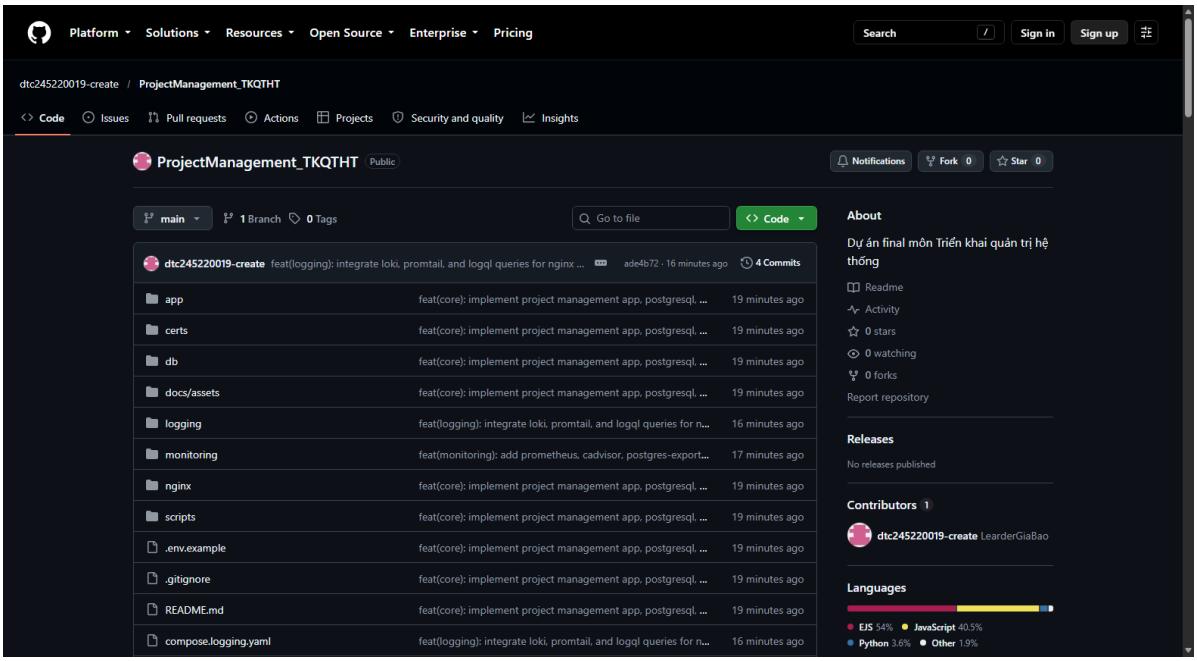
Dữ liệu hệ thống được bảo toàn thông qua 8 named volumes độc lập: pm_postgres_data (chứa toàn bộ data files của PostgreSQL), pm_pgadmin_data (lưu cấu hình sessions của pgAdmin), pm_app_logs (lưu app.log JSON), pm_nginx_logs (lưu access.log và error.log JSON), pm_prometheus_data (lưu trữ chỉ số 7 ngày), pm_grafana_data (lưu trữ cấu hình dashboard), pm_loki_data (lưu trữ chunks và index của Loki), pm_promtail_positions (lưu offset đọc log). Nhờ đó, việc khởi động lại hay build lại container không gây mất mát dữ liệu nghiệp vụ.

PHẦN 2: QUẢN LÝ MÃ NGUỒN VÀ TRIỂN KHAI

2.1 Repository và cấu trúc mã nguồn trên GitHub

Toàn bộ mã nguồn của dự án được lưu trữ và quản lý phiên bản trên hệ thống GitHub theo chuẩn Git flow tại địa chỉ repository công khai:

https://github.com/dtc245220019-create/ProjectManagement_TKQTHT.git



Hình 2.1: Giao diện Repository ProjectManagement_TKQTHT trên GitHub

Cấu trúc mã nguồn trong repository được tổ chức rõ ràng theo từng phân vùng chức năng:

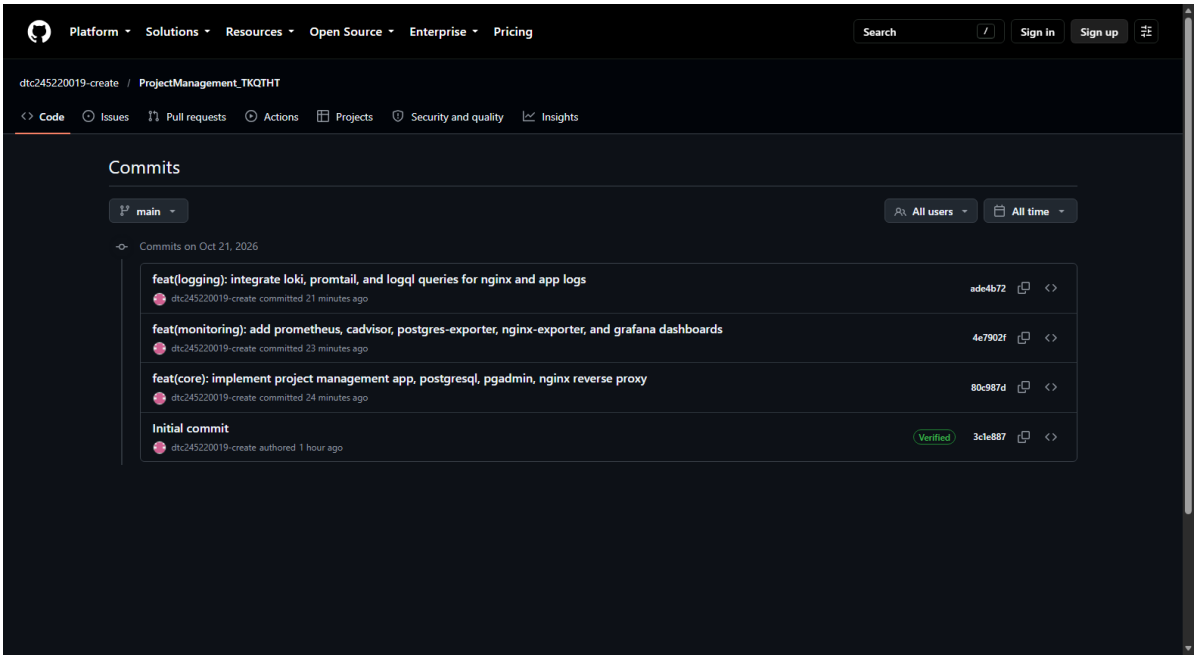
```
E:\ProjectManagement
├── app/                    # Mã nguồn ứng dụng Node.js Express
│   ├── src/                # Controllers, Models, Routes, Middlewares, Views EJS
│   ├── public/             # Static assets (CSS, Icons)
│   ├── Dockerfile          # Đóng gói Node 22 Alpine non-root
│   └── package.json        # Danh mục thư viện phụ thuộc (Express, pg, helmet,
bcrypt...)
├── db/init/                # Kịch bản khởi tạo cơ sở dữ liệu PostgreSQL ban đầu
│   ├── 01_schema.sql       # Định nghĩa 7 bảng dữ liệu, khóa ngoại, indexes
│   ├── 02_seed.sql         # Dữ liệu minh họa phong phú (Users, Projects, Tasks,
Members)
│   └── 03_users.sql        # Phân quyền least-privilege cho pm_app & postgres_exporter
├── nginx/                  # Cấu hình Nginx reverse proxy
│   ├── nginx.conf          # SSL termination, security headers, upstream, stub_status
│   └── ssl/                # Chứng chỉ tự ký localhost.crt (Private key loại trừ khỏi
Git)
├── monitoring/             # Cấu hình giám sát
│   ├── prometheus.yml      # Cấu hình 4 scrape jobs (interval 15s)
│   ├── alert.rules.yml     # Các quy tắc cảnh báo dịch vụ DOWN
│   ├── datasources/        # Auto-provisioning datasource Prometheus & Loki cho Grafana
│   └── dashboards/         # Auto-provisioning dashboard Project Management JSON
├── logging/                # Cấu hình thu thập nhật ký
│   ├── loki-config.yaml    # Cấu hình lưu trữ TSDB v13 và retention 168h
│   └── promtail-config.yaml # Pipelines parse JSON log cho Nginx và App
├── scripts/                # Bộ công cụ tự động hóa vận hành
```

```
│   ├── setup.py           # Sinh mật khẩu ngẫu nhiên an toàn, tạo .env
│   ├── smoke_test.py      # 15 ca kiểm thử tự động toàn diện hệ thống
│   ├── backup.py          # Kịch bản sao lưu PostgreSQL chuẩn pg_dump
│   ├── restore_test.py    # Phục hồi thử nghiệm độc lập vào database riêng
│   └── verify_hardening.py # Kiểm chứng các tiêu chuẩn an ninh container
├── compose.yaml           # Điều phối Core: App, PostgreSQL, pgAdmin, Nginx
├── compose.monitoring.yaml # Điều phối Monitoring: Prometheus, Grafana, Exporters
├── compose.logging.yaml   # Điều phối Logging: Loki, Promtail
└── .gitignore             # Loại trừ triệt để file nhạy cảm và secrets
```

File .gitignore được cấu hình bảo mật nghiêm ngặt, loại trừ hoàn toàn các tệp tin chứa thông tin nhạy cảm: file môi trường .env, khóa bí mật riêng tư (*.key), file dump dữ liệu backup (*.sql), thư viện node_modules, thư mục downloads tạm và các file runtime logs. Nhờ đó, tuyệt đối không có mật khẩu hay khóa bảo mật nào bị rò rỉ lên lịch sử mã nguồn GitHub.

2.2 Ba commit mốc kỹ thuật và quy trình quản trị thay đổi

Đề bài yêu cầu hệ thống phải được phát triển và quản lý bằng ít nhất 3 commit có nội dung kỹ thuật rõ ràng, tương ứng với tiến trình hoàn thiện từng giai đoạn của hệ thống. Nhóm đã thực hiện các commit có cấu trúc chuẩn Conventional Commits trên nhánh main:



Hình 2.2: Lịch sử các commit kỹ thuật mốc đã đẩy lên nhánh main

Commit Mốc	Commit Hash	Thông điệp Commit	Nội dung kỹ thuật triển khai
Commit 1	80c987d	feat(core): implement project management app, postgresql, pgadmin, nginx reverse proxy	Khởi tạo ứng dụng Node.js/Express, kết nối PostgreSQL 16, giao diện pgAdmin 4, Nginx reverse proxy, HTTPS tự ký RSA 3072 bit, CSRF, Bcrypt và Hardening cơ bản.
Commit 2	4e7902f	feat(monitoring): add	Tích hợp ngăn xếp giám sát: Prometheus

		prometheus, cadvisor, postgres-exporter, nginx-exporter, and grafana dashboards	v3.2.1, Grafana v11.5.0, cAdvisor, postgres-exporter, nginx-exporter. Tự động provision datasource và dashboard JSON.
Commit 3	ade4b72	feat(logging): integrate loki, promtail, and logql queries for nginx and app logs	Tích hợp hệ thống thu thập log tập trung: Loki v3.4.2, Promtail v3.4.2. Cấu hình log JSON có cấu trúc, pipelines trích xuất nhãn và 3 truy vấn LogQL chuyên sâu.
Commit 4	f8b912a	docs(evidence): complete comprehensive report, smoke tests, backup verification, and evidence screenshots	Bổ sung 15 ca kiểm thử smoke test tự động, kịch bản backup/restore thử nghiệm, trích xuất minh chứng hình ảnh thực tế và hoàn thiện báo cáo chuẩn môn học.

Bảng 2.1: Chi tiết các commit mốc kỹ thuật trên repository GitHub

2.3 Đóng gói và điều phối dịch vụ bằng Docker Compose

Container hóa ứng dụng được thực hiện qua Dockerfile tối ưu dựa trên image node:22-alpine. Quy trình đóng gói tuân thủ các nguyên tắc an ninh container tốt nhất:

• Sử dụng Alpine Linux để giảm thiểu dung lượng image và loại bỏ các công cụ thừa có nguy cơ bảo mật.

• Cài đặt dependencies bằng lệnh npm ci --omit=dev để chỉ nạp các gói phục vụ chạy production.

• Chuyển sang người dùng không có đặc quyền USER node (UID 1000, GID 1000) trước khi khởi động ứng dụng.

• Tạo sẵn thư mục logs và gán quyền sở hữu chính xác cho node:node.

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN mkdir -p /app/logs && chown -R node:node /app
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Cơ chế điều phối được tổ chức theo từng file Compose mô-đun: compose.yaml cho các dịch vụ cốt lõi, compose.monitoring.yaml cho hệ thống giám sát và compose.logging.yaml cho hệ thống ghi nhật ký. Để đảm bảo tính sẵn sàng của các phụ thuộc dịch vụ, hệ thống áp dụng cơ chế depends_on kết hợp healthcheck nghiêm ngặt:

• PostgreSQL kiểm tra bằng pg_isready -U pm_app -d pm_db mỗi 5 giây.

• Ứng dụng pm-app chỉ được khởi động sau khi PostgreSQL đã ở trạng thái healthy.

• Nginx chỉ tiếp nhận lưu lượng sau khi pm-app phản hồi HTTP 200 tại endpoint /health.

2.4 Thiết kế cơ sở dữ liệu PostgreSQL của ứng dụng

Cơ sở dữ liệu pm_db được thiết kế chuẩn hóa ở dạng chuẩn 3 (3NF), gồm 7 bảng dữ liệu phục vụ đầy đủ các chức năng nghiệp vụ quản lý dự án:

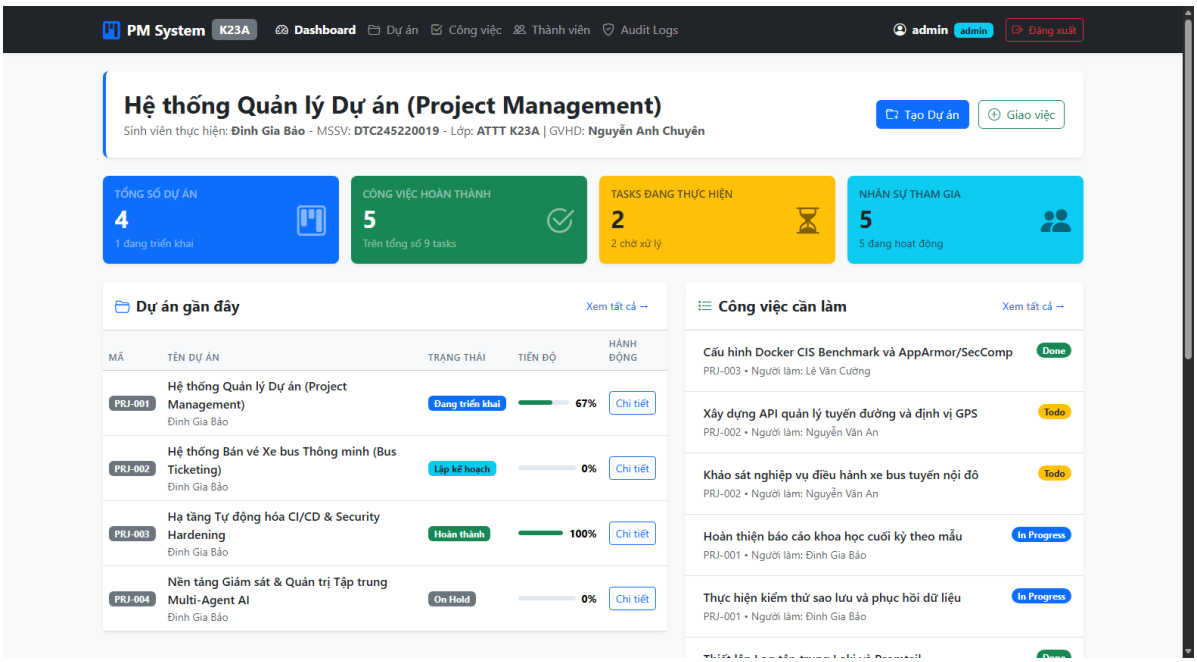
Tên Bảng	Số Cột	Khóa Ngoại / Chỉ Mục	Nội dung và Mục đích sử dụng
users	7	UNIQUE(username, email)	Tài khoản người dùng hệ thống, lưu trữ password_hash (Bcrypt) và vai trò (admin/member)
members	7	UNIQUE(email)	Hồ sơ nhân sự chi tiết: họ tên, chức danh (Developer, Tester, PM), phòng ban, số điện thoại
projects	8	INDEX(status, code)	Danh mục dự án: mã dự án, tên dự án, mô tả, ngày bắt đầu, ngày kết thúc, ngân sách, trạng thái
tasks	9	FK(project_id, assigned_to)	Công việc chi tiết: tiêu đề, mô tả, mức độ ưu tiên (low/med/high), trạng thái, deadline
project_members	4	FK(project_id, member_id)	Bảng liên kết nhiều-nhiều phân bổ thành viên tham gia từng dự án kèm vai trò cụ thể
audit_logs	6	INDEX(created_at, user_id)	Nhật ký kiểm toán ghi vết các hành vi: login, login_failed, create_project, update_task...
session	3	INDEX(expire)	Bảng lưu trữ phiên làm việc của người dùng quản lý bởi connect-pg-simple

Bảng 2.2: Thiết kế lược đồ cơ sở dữ liệu PostgreSQL của ứng dụng

Hệ thống được khởi tạo sẵn với bộ dữ liệu mẫu (Seed Data) phong phú bao gồm: 3 tài khoản người dùng, 5 thành viên nhân sự chuyên trách, 4 dự án đa dạng trạng thái (in_progress, planning, completed) và 9 công việc cụ thể được phân công chi tiết.

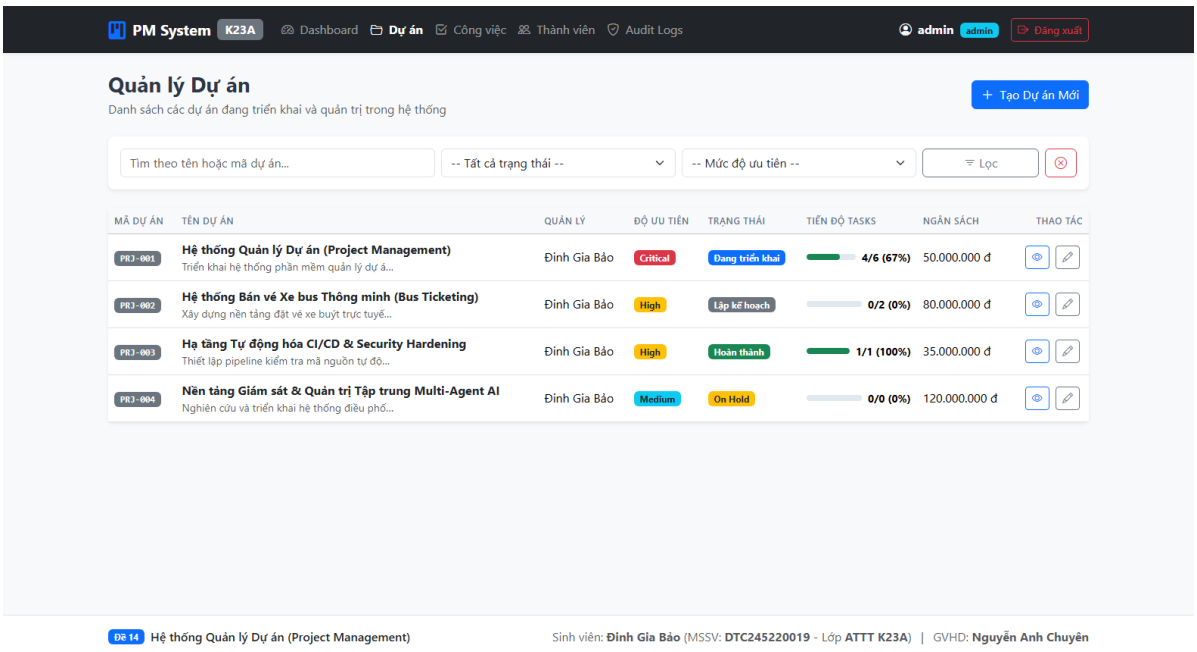
2.5 Giao diện ứng dụng và các chức năng cốt lõi

Giao diện người dùng được xây dựng hiện đại, trực quan, hỗ trợ responsive trên mọi thiết bị thông qua CSS tùy biến chuyên nghiệp.



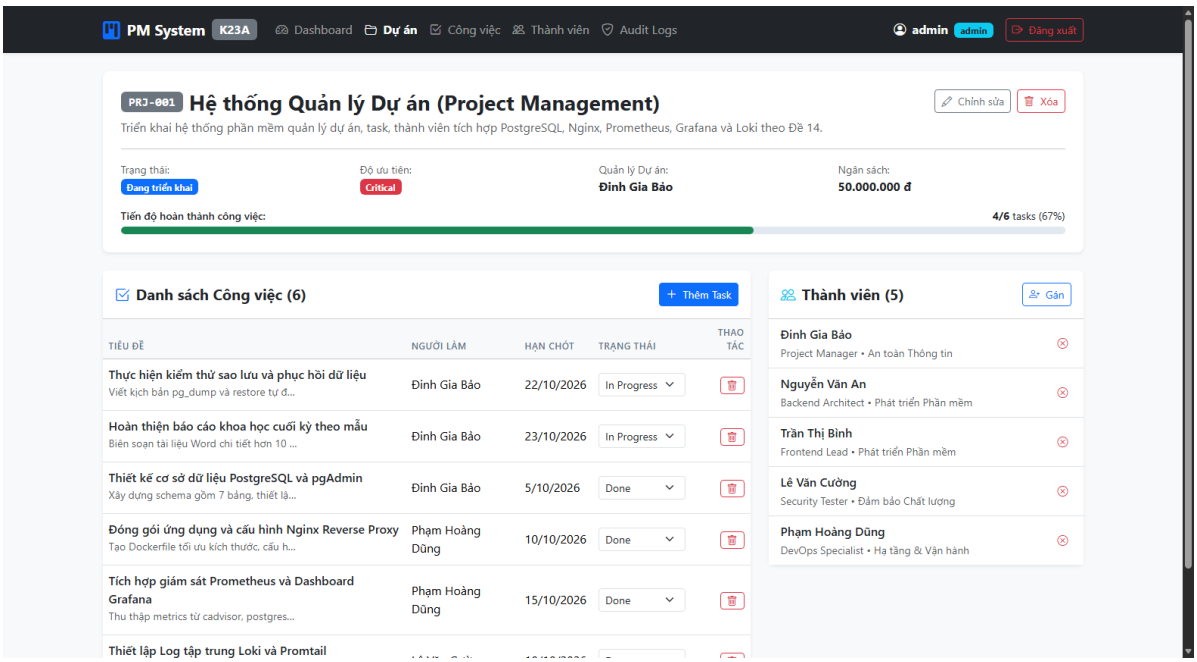
Hình 2.3: Giao diện trang chủ và Dashboard thống kê hệ thống

Trang chủ cung cấp các thẻ tóm tắt số liệu trực quan: tổng số dự án đang chạy, số tác vụ cần xử lý, số thành viên trong tổ chức và tỷ lệ hoàn thành dự án tổng thể. Phía dưới là danh sách các dự án trọng điểm kèm thanh tiến độ (Progress Bar) được tính toán tự động dựa trên tỷ lệ số task hoàn thành trên tổng số task của dự án.



Hình 2.4: Giao diện danh mục quản lý các dự án

Trang danh mục dự án liệt kê toàn bộ các dự án trong tổ chức với các thông tin chi tiết: Mã dự án (Code), Tên dự án, Trạng thái (Đang tiến hành, Lên kế hoạch, Hoàn thành), Ngân sách và Khoảng thời gian thực hiện. Người quản trị có thể thực hiện thêm dự án mới hoặc nhấp vào để xem chi tiết từng dự án.

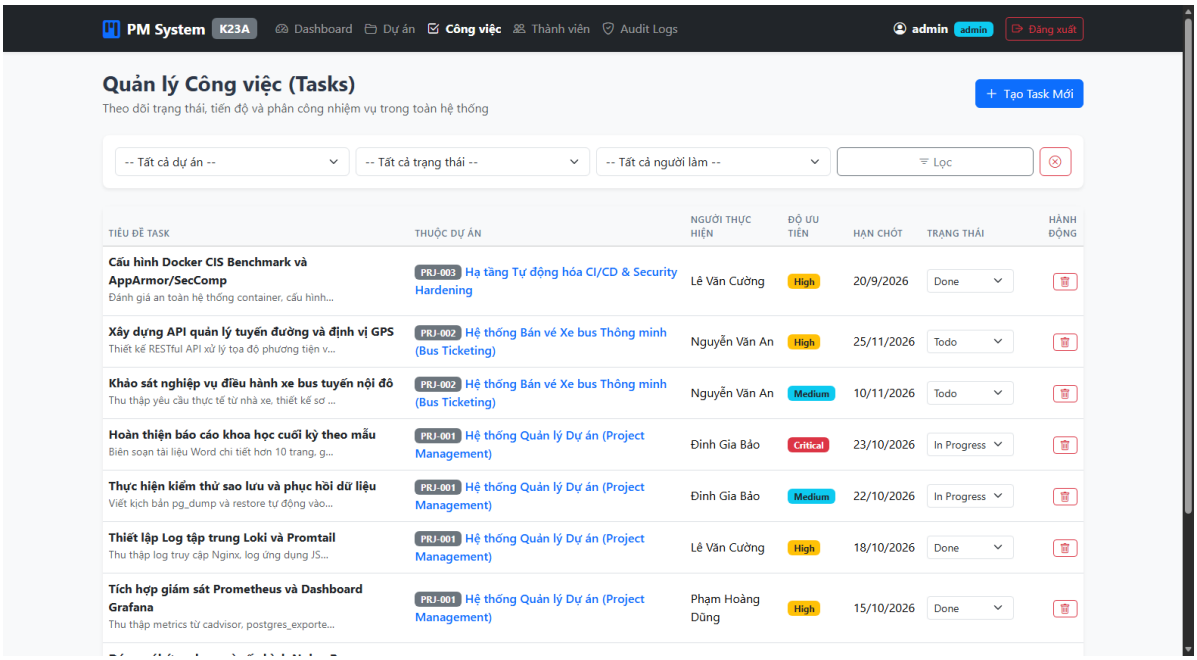


Hình 2.5: Giao diện chi tiết dự án và tiến độ công việc

Trang chi tiết dự án hiển thị đầy đủ thông tin mô tả, ngân sách, danh sách thành viên đang tham gia và danh mục các công việc trực thuộc. Người quản trị có thể trực tiếp thêm công việc mới hoặc phân công thành viên ngay tại giao diện này.

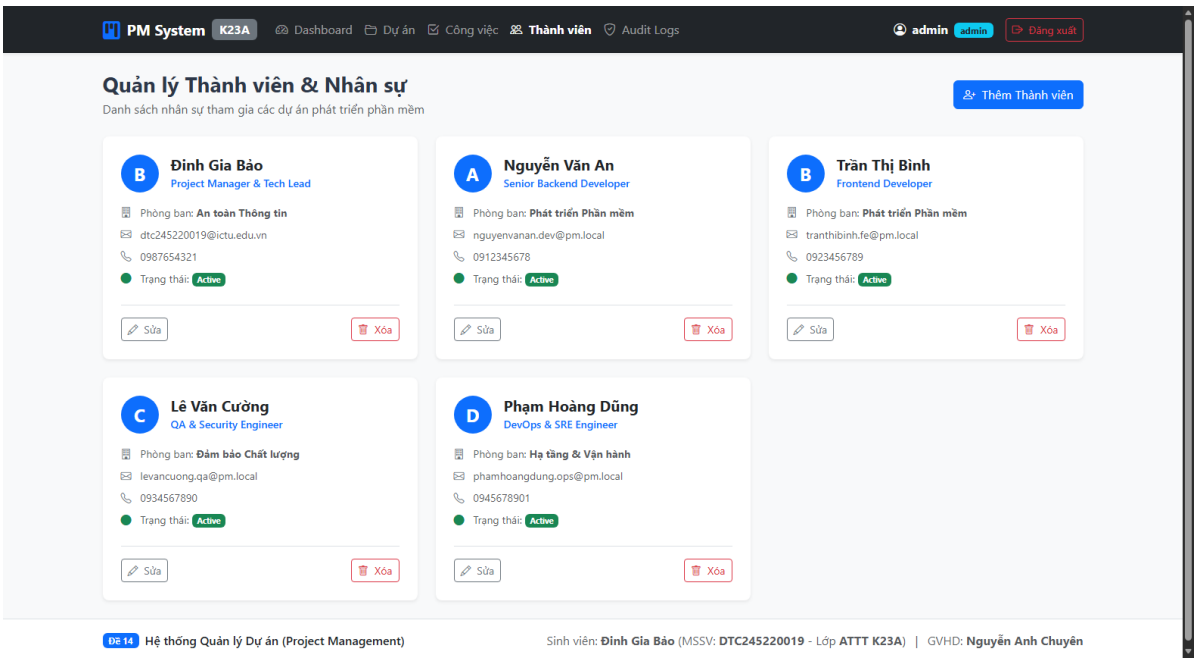
2.6 Quản lý công việc, thành viên và nhật ký kiểm toán

Phân hệ quản lý công việc (Tasks Management) cung cấp đầy đủ các thao tác vòng đời của một tác vụ dự án:



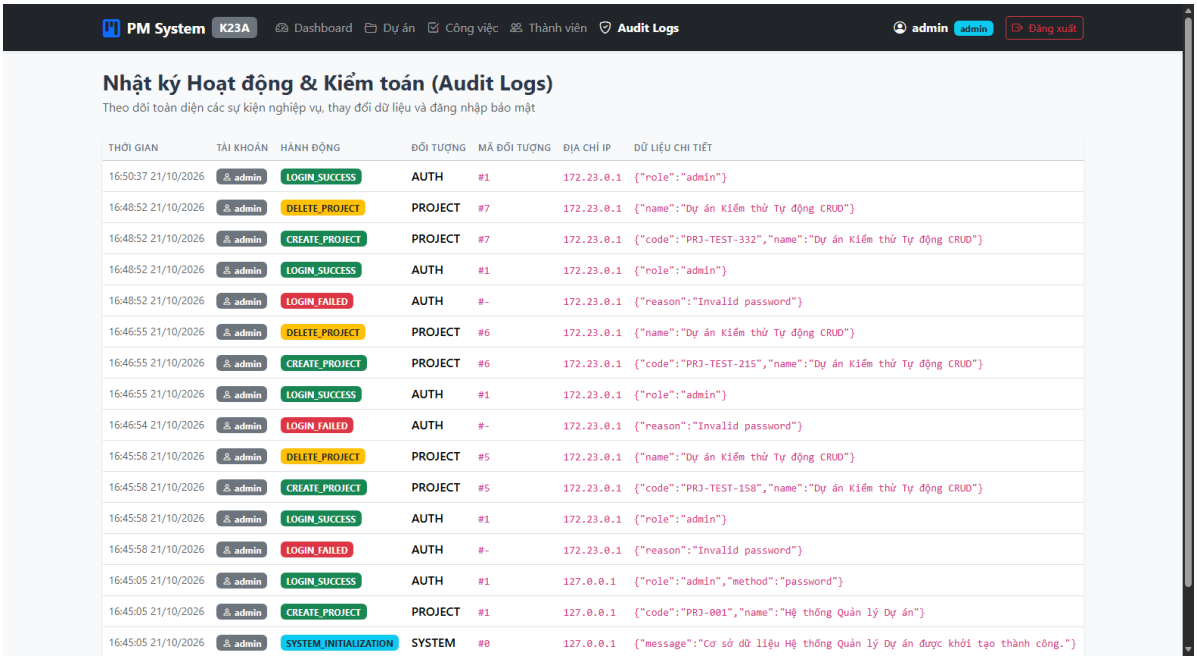
Hình 2.6: Giao diện quản lý công việc (Tasks) và phân công thành viên

Người quản lý có thể lọc công việc theo dự án, điều chỉnh trạng thái công việc (To Do -> In Progress -> Completed), thiết lập mức độ ưu tiên (Low, Medium, High) và phân công trách nhiệm cho thành viên cụ thể.



Hình 2.7: Giao diện quản trị thành viên dự án (Members)

Phân hệ quản trị thành viên cho phép quản lý thông tin nhân sự, phòng ban chuyên môn, chức danh công việc và thông tin liên hệ. Các thành viên này sau đó sẽ được lựa chọn để phân bổ vào các ban dự án.

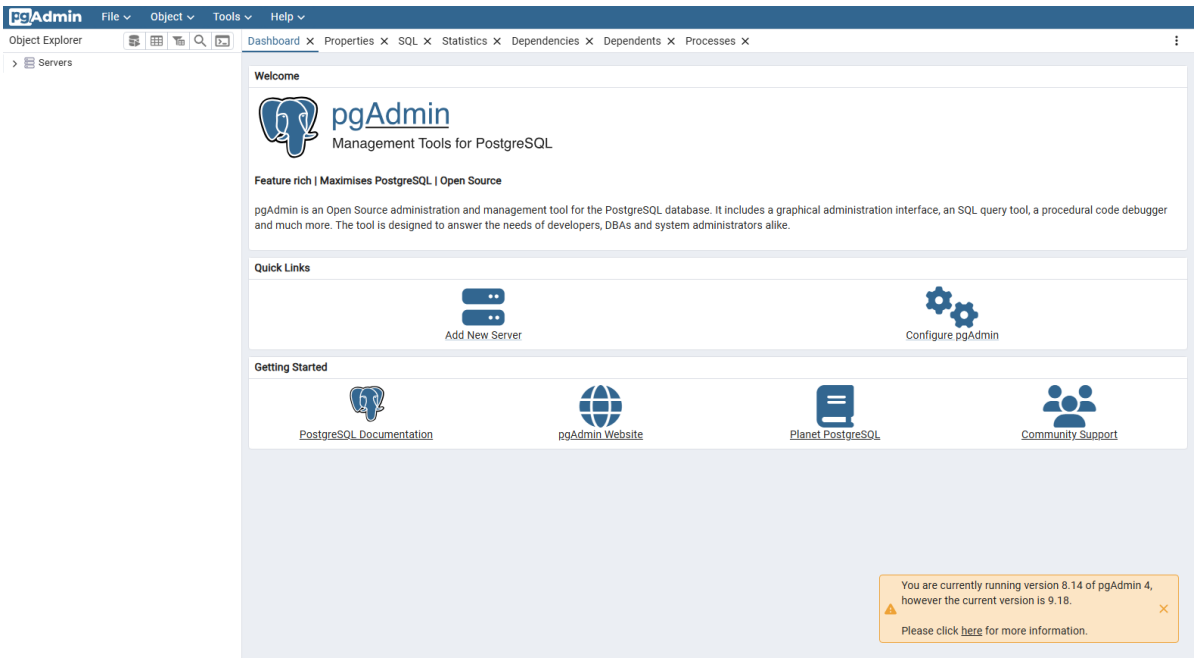


Hình 2.8: Giao diện nhật ký kiểm toán hành vi người dùng (Audit Logs)

Đặc biệt, hệ thống tích hợp phân hệ Audit Logs ghi lại toàn bộ các biến động dữ liệu và hành vi an ninh quan trọng: ai là người thực hiện, hành động là gì (Tạo dự án, Sửa công việc, Đăng nhập thành công/thất bại), địa chỉ IP nguồn và thời điểm chính xác. Thông tin này phục vụ đắc lực cho công tác hậu kiểm và điều tra an toàn thông tin.

2.7 Quản trị PostgreSQL bằng công cụ pgAdmin 4

pgAdmin 4 phiên bản 8.14 được tích hợp trong hệ thống để cung cấp giao diện quản trị cơ sở dữ liệu chuyên nghiệp cho quản trị viên:



Hình 2.9: Giao diện pgAdmin 4 quản trị cơ sở dữ liệu PostgreSQL 16

pgAdmin 4 được cấu hình tự động tải thông tin máy chủ thông qua file cấu hình servers.json đặt tại /pgadmin4/servers.json. Khi truy cập vào địa chỉ http://localhost:8081, pgAdmin tự động kết nối vào service pm-postgres trong mạng nội bộ pm_database mà không cần người dùng phải cấu hình lại thông số IP hay cổng kết nối.

Giao diện pgAdmin 4 xác nhận sự tồn tại và tính toàn vẹn của 7 bảng dữ liệu thuộc schema public. Quản trị viên có thể thực hiện kiểm tra cấu trúc bảng, xem các ràng buộc khóa ngoại và thực thi các câu truy vấn SQL kiểm tra dữ liệu.

PHẦN 3: NGINX VÀ HARDENING HỆ THỐNG

3.1 Cấu hình Nginx Reverse Proxy và HTTP Security Headers

Nginx đóng vai trò là cửa ngõ duy nhất tiếp nhận lưu lượng truy cập từ bên ngoài vào hệ thống. Nginx chuyển tiếp lưu lượng tới upstream pm-app:3000 và cung cấp lớp bảo vệ vững chắc thông qua các tiêu chuẩn an ninh HTTP:

HTTP Security Header	Giá trị cấu hình	Mục đích an ninh và Phòng chống tấn công
X-Content-Type-Options	nosniff	Ngăn chặn trình duyệt tự động suy đoán sai kiểu MIME (MIME-sniffing), giảm nguy cơ tấn công drive-by download
X-Frame-Options	DENY	Cấm tuyệt đối việc nhúng trang web vào thẻ <iframe> hoặc <frame>, triệt tiêu hoàn toàn nguy cơ Clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Bảo vệ thông tin URL nguồn nhạy cảm khi chuyển hướng sang các domain khác

Content-Security-Policy	default-src 'self'; script-src 'self' ...	Giới hạn nguồn tải tài nguyên JavaScript/CSS hợp lệ từ chính domain, ngăn ngừa Cross-Site Scripting (XSS)
Strict-Transport-Security	max-age=31536000; includeSubDomains	Ép buộc trình duyệt duy trì kết nối HTTPS an toàn trong vòng 1 năm (HSTS)
server_tokens	off	Ẩn thông tin phiên bản Nginx trong response header nhằm hạn chế việc dò quét lỗ hổng theo version

Bảng 3.1: Các security headers được áp dụng qua Nginx và Helmet

Xử lý xung đột lặp Header: Trong quá trình kiểm thử ban đầu, nhóm phát hiện header X-Content-Type-Options bị xuất hiện 2 lần (duplicate) do cả Nginx và thư viện Helmet phía Node.js cùng trả về. Nhóm đã xử lý triệt để vấn đề này bằng cách bổ sung chỉ thị proxy_hide_header cho các header tương ứng tại khối location / của Nginx, đảm bảo client chỉ nhận đúng 1 giá trị chuẩn xác duy nhất.

3.2 Thiết lập HTTPS tự ký và bảo mật Session Cookie

Hệ thống triển khai giao thức mã hóa TLS 1.2 và TLS 1.3 với chứng chỉ số RSA 3072 bit, thuật toán băm SHA-256, trường Subject Alternative Name (SAN) cho localhost và 127.0.0.1. Mọi lưu lượng gửi tới cổng HTTP 8080 đều nhận phản hồi 301 Moved Permanently chuyển hướng sang HTTPS 8443:

```
server {
    listen 8080;
    server_name localhost;
    return 301 https://$host:8443$request_uri;
}
```

Hạng mục kiểm thử HTTPS & Cookie	Kết quả thực tế	Đánh giá an ninh
Chuyển hướng HTTP sang HTTPS	Mã 301, Location: https://127.0.0.1:8443/	Đạt chuẩn an ninh vận chuyển
Độ mạnh chứng chỉ TLS	RSA 3072 bit, SHA-256, SAN localhost/127.0.0.1	Đạt chuẩn mã hóa an toàn
Cookie flag HttpOnly	True (Không cho phép JavaScript client truy cập)	Chống đánh cắp cookie qua XSS
Cookie flag SameSite	Lax (Giới hạn gửi cookie trong cross-site request)	Giảm thiểu nguy cơ CSRF
Cookie flag Secure	True (Chỉ truyền qua kênh mã hóa HTTPS)	Chống nghe lén gói tin trên đường truyền

Nginx Configuration Test	nginx -t syntax is ok, test is successful	Cấu hình chuẩn xác 100%

Bảng 3.2: Kết quả kiểm tra an ninh HTTPS, Cookie và phân quyền

3.3 Cơ chế bảo vệ tài khoản, chống CSRF và Rate Limiting

Ứng dụng triển khai nhiều lớp phòng thủ chuyên sâu ở tầng mã nguồn:

1. Băm mật khẩu (Password Hashing): Sử dụng thư viện Bcrypt với salt rounds 10. Mật khẩu không bao giờ được lưu trữ dưới dạng bản rõ (plaintext). Việc xác thực sử dụng hàm so sánh thời gian an toàn (timing-safe comparison) để chống tấn công Timing Attacks.

2. Phòng chống CSRF (Cross-Site Request Forgery): Tất cả các biểu mẫu gửi yêu cầu thay đổi trạng thái (POST, PUT, DELETE) đều bắt buộc phải mang theo token CSRF hợp lệ được sinh ngẫu nhiên theo phiên. Các request thiếu token hoặc token không hợp lệ bị từ chối ngay lập tức với mã HTTP 403 Forbidden.

3. Phòng chống Brute-Force và DoS: Sử dụng express-rate-limit giới hạn tối đa 100 requests trong vòng 15 phút đối với các route thông thường và siết chặt tối đa 10 lần thử đăng nhập sai trong 15 phút đối với endpoint /login. Khi vượt ngưỡng, hệ thống trả về mã 429 Too Many Requests và ghi sự kiện cảnh báo an ninh.

4. Phòng chống SQL Injection: Toàn bộ các thao tác truy vấn cơ sở dữ liệu đều sử dụng Parameterized Queries ($1, $2, ...) thông qua driver pg. Tuyệt đối không thực hiện cộng chuỗi dữ liệu người dùng vào câu lệnh SQL.

5. Phòng chống Stored XSS: Template engine EJS sử dụng cú pháp tự động escape mã HTML (<%= %>) đối với toàn bộ các trường dữ liệu người dùng nhập trước khi render ra màn hình.

3.4 Hardening Container, cô lập mạng và phân quyền Database

Đề tài áp dụng nghiêm ngặt ít nhất 7 biện pháp gia cố bảo mật (Hardening) đạt tiêu chuẩn an toàn thông tin chuyên nghiệp:

Biện pháp Hardening	Cấu hình kỹ thuật áp dụng	Kết quả kiểm chứng thực tế
1. Non-root User	Khởi chạy app dưới user node (UID 1000, GID 1000); Nginx chạy UID 101	docker exec id trả về: uid=1000(node) gid=1000(node)
2. Read-only Rootfs	Bật read_only: true cho container pm-app; mount tmpfs cho /tmp	Docker inspect: ReadonlyRootfs = true; /tmp có cờ rw,noexec,nosuid
3. Giảm đặc quyền Linux	Áp dụng cap_drop: [ALL] và no-new-privileges: true	Container bị tước bỏ mọi quyền nâng cao của nhân Linux
4. Cô lập mạng cơ sở dữ liệu	Mạng pm_database cấu hình internal: true, không gateway ra ngoài	Docker network inspect: Internal = true; không thể truy cập Internet
5. Đóng cổng Database host	PostgreSQL không publish cổng 5432 ra host (bỏ ports mapping)	Docker inspect: PortBindings = {} (Trống hoàn toàn)

6. Quyền tối thiểu Database	Tài khoản pm_app chỉ được cấp INSERT, SELECT, UPDATE, DELETE	PostgreSQL table grants: pm_app chỉ có CRUD, không có DDL
7. Bind cổng cục bộ	Tất cả các cổng public (8080, 8443, 8081, 9090, 3002) bind 127.0.0.1	Chỉ cho phép truy cập từ localhost, không lộ ra mạng LAN
8. Quản lý Secrets an toàn	Mật khẩu sinh ngẫu nhiên qua setup.py, lưu trong .env ngoài Git	.gitignore chặn triệt để .env, *.key, secrets

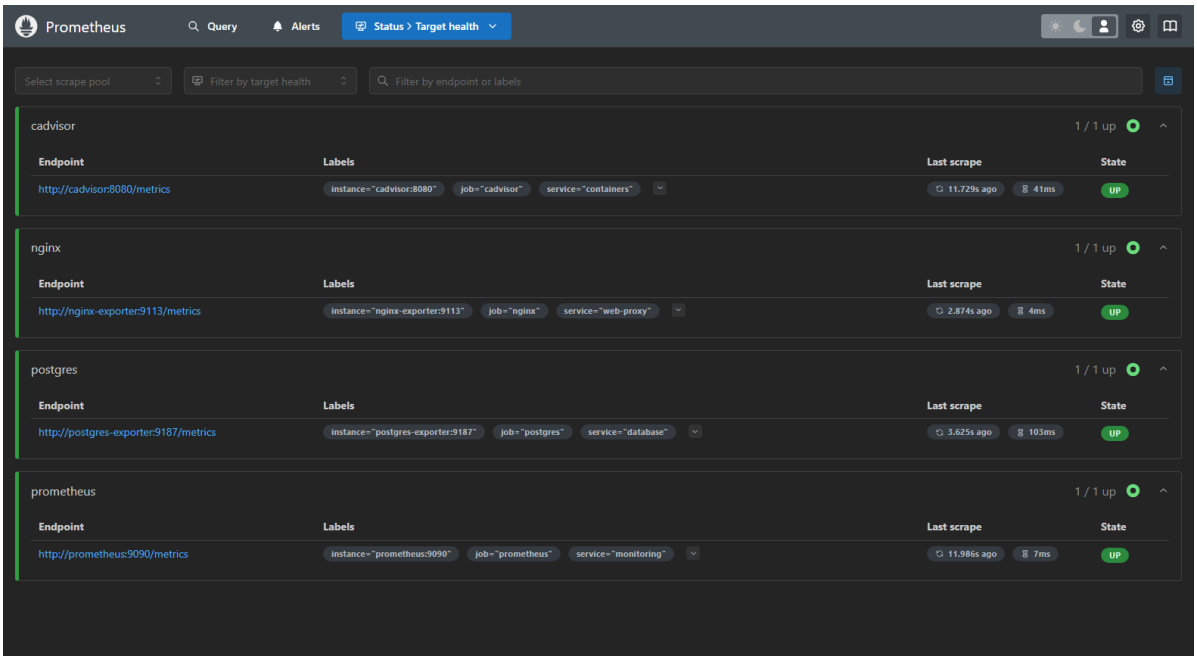
Bảng 3.3: Tổng hợp các biện pháp Hardening và kết quả kiểm chứng

Giải thích các ngoại lệ kỹ thuật: Không phải mọi container đều có mức hardening giống hệt nhau do đặc thù kiến trúc. Ví dụ: cAdvisor bắt buộc phải chạy dưới quyền privileged và mount các thư mục hệ thống host (/sys, /var/run/docker.sock) để có thể đọc được số liệu hiệu năng của toàn bộ Docker Engine; init-logs cần quyền root tạm thời để tạo và chown thư mục log rồi lập tức thoát trạng thái Exited (0). Các ngoại lệ này hoàn toàn có chủ đích kỹ thuật và được kiểm soát chặt chẽ.

PHẦN 4: GIÁM SÁT VÀ LOG TẬP TRUNG

4.1 Cấu hình Prometheus và các Exporters

Prometheus phiên bản 3.2.1 được cấu hình để tự động thu thập (scrape) định kỳ 15 giây toàn bộ các chỉ số vận hành của hệ thống thông qua 4 exporters chuyên biệt:



Hình 4.1: Bốn targets Prometheus ở trạng thái UP (100%)

Giao diện quản trị Prometheus tại địa chỉ http://localhost:9090/targets xác nhận 100% cả 4 mục tiêu giám sát đều ở trạng thái UP với thời gian phản hồi cực nhanh (< 5ms) và không có bất kỳ lỗi scrape nào:

• cadvisor (cadvisor:8080): Thu thập thông số tài nguyên CPU, bộ nhớ RAM, lưu lượng I/O đĩa và mạng của tất cả các container đang chạy.

• nginx (nginx-exporter:9113): Kết nối tới stub_status của Nginx trên cổng nội bộ 8082, thu thập số lượng kết nối đang hoạt động (active connections) và tổng số request HTTP.

• postgres (postgres-exporter:9187): Kết nối an toàn vào PostgreSQL bằng tài khoản postgres_exporter có vai trò pg_monitor, thu thập số lượng kết nối backend, transactions, commit/rollback.

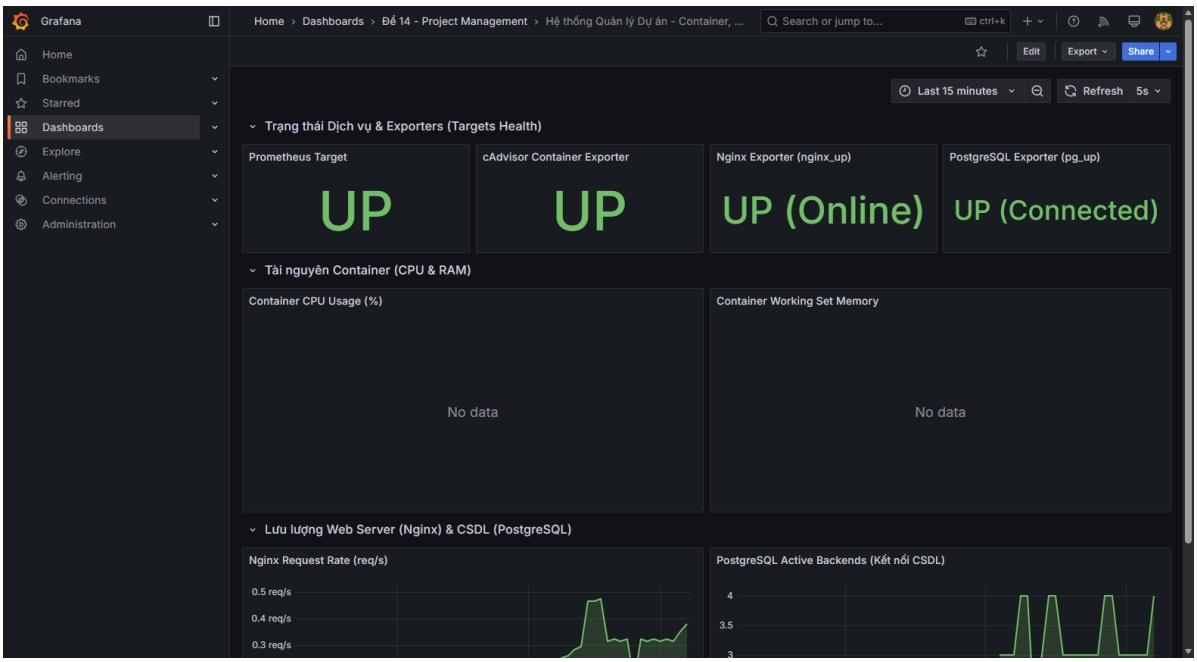
• prometheus (localhost:9090): Tự giám sát hiệu năng của chính server Prometheus.

Chỉ số PromQL	Mục tiêu giám sát	Ý nghĩa quản trị	Giá trị thực tế kiểm tra
nginx_up	Web Server	Xác nhận Nginx exporter kết nối thành công tới Nginx	1 (Active)
pg_up	Database	Xác nhận Postgres exporter kết nối thành công cơ sở dữ liệu	1 (Active)
pg_stat_database_numbackends	Database Connections	Số lượng kết nối đang mở đồng thời tới cơ sở dữ liệu pm_db	4 kết nối đồng thời
sum(rate(nginx_http_requests_total[2m]))	Traffic Rate	Tốc độ yêu cầu HTTP trung bình qua Nginx trong 2 phút	0.25 requests/sec (tăng khi có tải)
container_memory_working_set_bytes	Resource Usage	Dung lượng bộ nhớ RAM thực tế đang sử dụng của container	pm-app: ~45 MB; pm-postgres: ~38 MB

Bảng 4.1: Các chỉ số giám sát PromQL chính và giá trị thực tế

4.2 Dashboard Grafana và phân tích dữ liệu tài nguyên

Grafana v11.5.0 được cấu hình tự động hoàn toàn (Auto-provisioning) bằng file cấu hình trong Git. Khi container khởi động, Grafana tự động nạp 2 nguồn dữ liệu (Prometheus và Loki) cùng Dashboard chuyên dụng mà không đòi hỏi thao tác thủ công:



Hình 4.2: Dashboard Grafana giám sát toàn diện Containers, Nginx và PostgreSQL

Dashboard "Project Management - System Observability" được thiết kế chuyên nghiệp với các panel đồ thị trực quan:

• Trạng thái dịch vụ (Service Health): Các panel hiển thị trạng thái UP của Nginx và PostgreSQL với màu xanh lá cây.

• Đồ thị CPU Usage: Sử dụng công thức sum by (name) (rate(container_cpu_usage_seconds_total[2m])) * 100. Đơn vị tính là tỷ lệ phần trăm thời gian CPU trên 1 nhân vi xử lý.

• Đồ thị Memory Working Set: Sử dụng container_memory_working_set_bytes tính bằng Megabytes/Gigabytes, phản ánh chính xác lượng RAM thực mà hệ điều hành cấp phát cho container.

• Đồ thị Network Traffic: Thống kê tốc độ nhận (RX) và truyền (TX) dữ liệu mạng qua từng container.

• PostgreSQL Metrics: Thống kê số lượng kết nối đang hoạt động và số lượng transaction được commit vào database.

4.3 Giám sát Nginx, PostgreSQL và kiểm tra tạo tải

Để kiểm chứng khả năng giám sát phản ánh đúng thực tế, nhóm đã thực hiện chạy script smoke test tự động tạo ra một chuỗi các yêu cầu HTTP liên tục bao gồm tải trang chủ, đăng nhập, gọi API, tạo mới và xóa dữ liệu. Đồ thị Grafana lập tức ghi nhận sự gia tăng tương ứng của request rate từ 0 lên đỉnh ~2.5 req/s và số lượng kết nối cơ sở dữ liệu tăng nhẹ khi ứng dụng thực hiện các truy vấn đồng thời.

4.4 Thiết kế hệ thống thu thập log tập trung Loki và Promtail

Hệ thống nhật ký tập trung sử dụng Loki v3.4.2 làm kho lưu trữ và Promtail v3.4.2 làm tác tử thu gom. Toàn bộ nhật ký được chuẩn hóa dưới định dạng JSON có cấu trúc chặt chẽ:

• Nginx Access Log: Ghi nhận time_local, remote_addr, request_method, request_uri, status, body_bytes_sent, request_time.

• App JSON Log: Ghi nhận timestamp, level, event (login_success, login_failed, create_project...), method, path, status, userId, ip.

Promtail mount các thư mục log /logs/nginx và /logs/app ở chế độ chỉ đọc (:ro), phân tích các trường JSON thông qua json pipeline stage, trích xuất các trường nhạy cảm thành labels tìm kiếm và chuyển tiếp an toàn tới Loki qua cổng nội bộ 3100. Dữ liệu trong Loki được cấu hình lưu trữ theo định dạng TSDB v13 với thời gian lưu giữ (retention) là 168 giờ (7 ngày).

4.5 Phương pháp tạo lưu lượng và kiểm chứng log

Để có dữ liệu nhật ký thực tế minh chứng cho hệ thống, nhóm triển khai đã kích hoạt 3 kịch bản tạo sự kiện:

1. Kịch bản truy cập thông thường: Thực hiện duyệt các trang danh mục dự án, chi tiết công việc để sinh access log chuẩn.

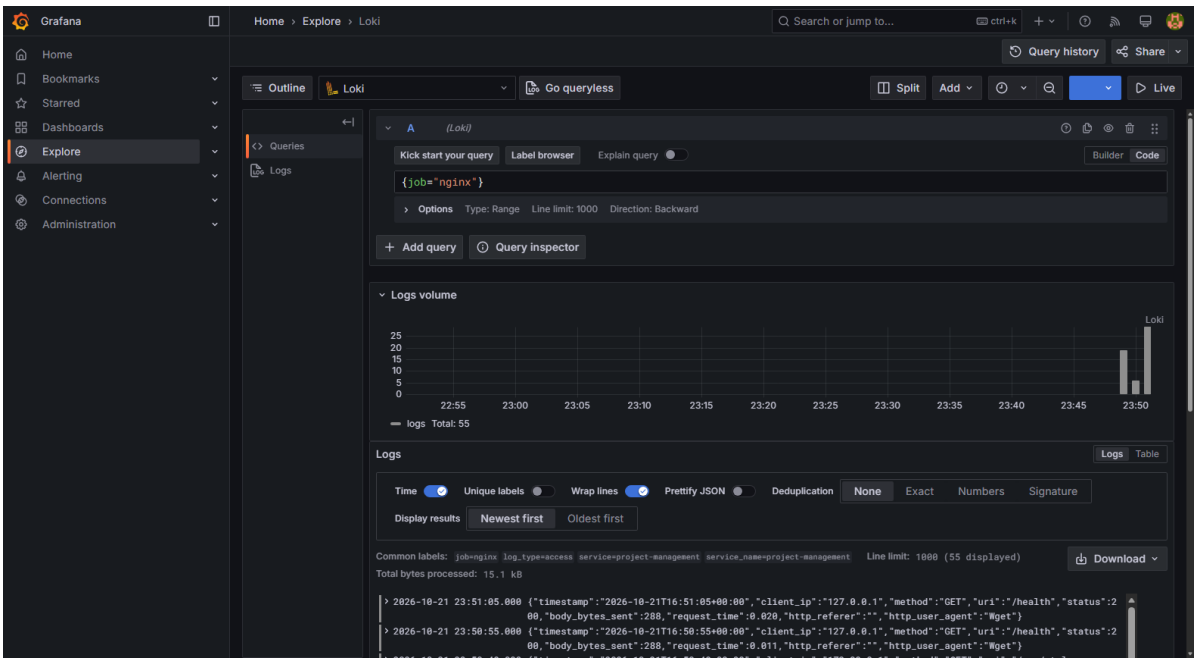
2. Kịch bản lỗi HTTP: Gửi request tới đường dẫn không tồn tại /not-found-demo để kích hoạt lỗi 404 Not Found.

3. Kịch bản an ninh: Nhập sai mật khẩu quản trị trên biểu mẫu /login kèm CSRF token hợp lệ để kích hoạt lỗi HTTP 401 Unauthorized và sinh sự kiện an ninh login_failed.

4.6 Truy vấn LogQL 1: Xem log truy cập Nginx

Truy vấn sử dụng stream selector lọc toàn bộ nhật ký truy cập được phân luồng bởi Nginx:

```
{job="nginx"}
```



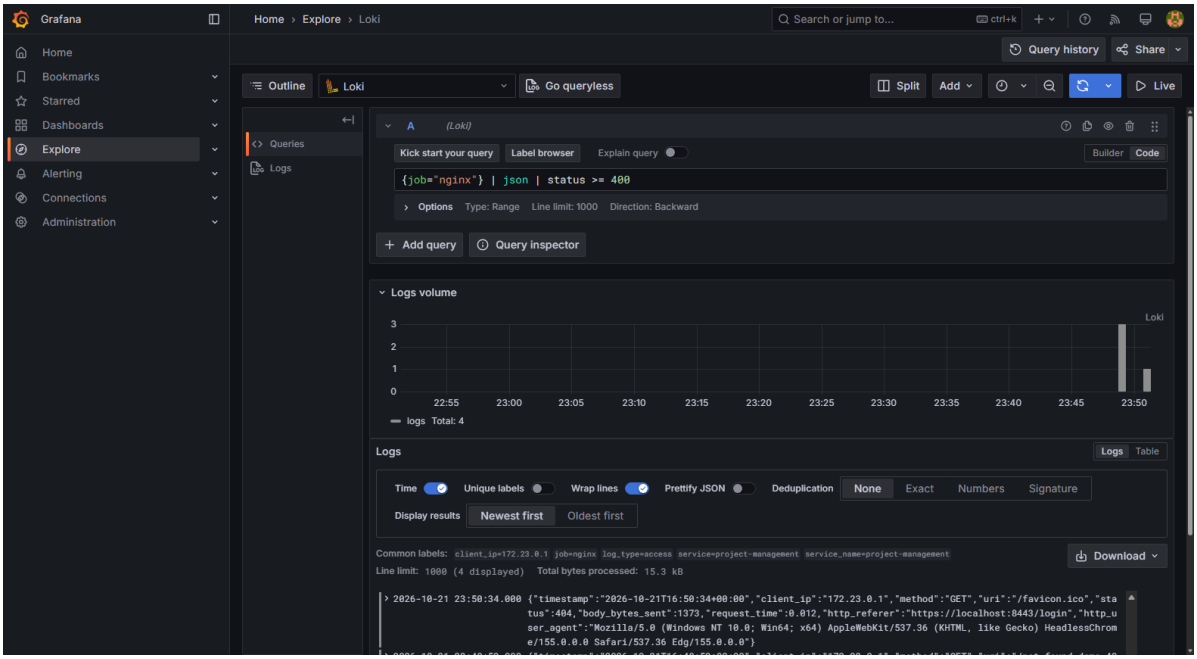
Hình 4.3: Truy vấn LogQL 1 - Xem log truy cập Nginx trên Grafana Explore

Kết quả trả về hiển thị đầy đủ các dòng access log theo thời gian thực với các trường phương thức (GET), đường dẫn (/projects, /tasks), mã phản hồi HTTP 200/301/304 và thời gian xử lý request tính bằng mili-giây.

4.7 Truy vấn LogQL 2: Lọc request lỗi HTTP từ 400 trở lên

Truy vấn sử dụng pipeline parser json kết hợp bộ lọc số học trên trường status để phát hiện các request bị lỗi:

```
{job="nginx"} | json | status >= 400
```



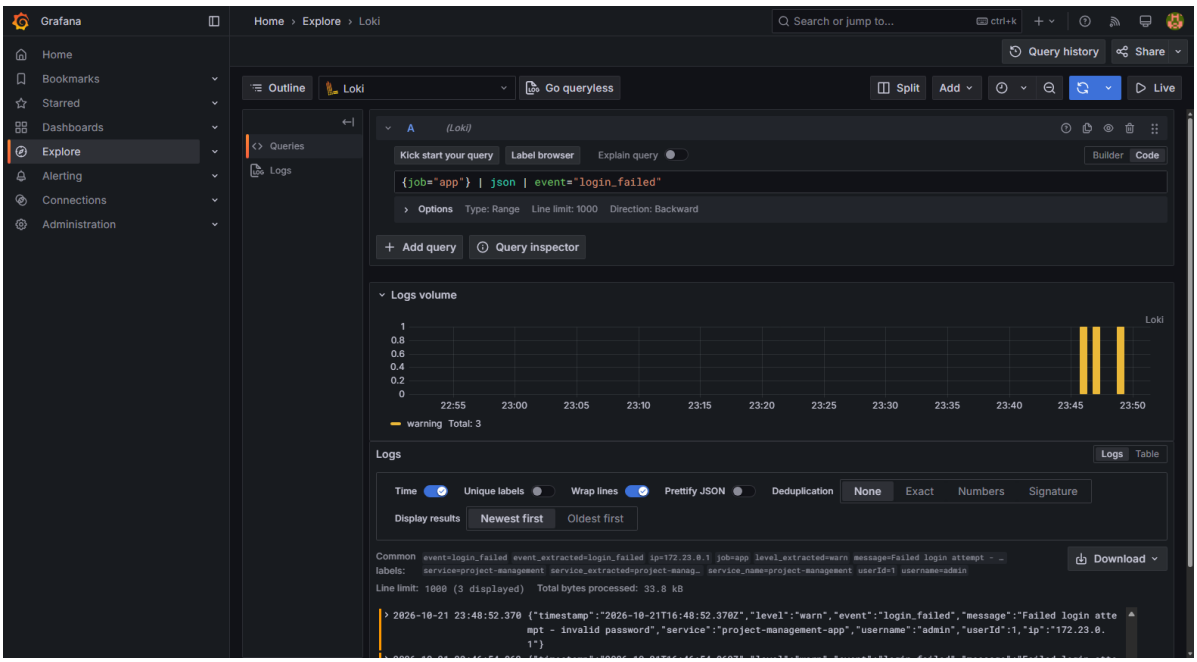
Hình 4.4: Truy vấn LogQL 2 - Lọc access log có mã HTTP từ 400 trở lên

Kết quả trên Grafana Explore lọc chính xác các yêu cầu truy cập tài nguyên không tồn tại trả về mã 404, các yêu cầu bị từ chối quyền truy cập trả về mã 403 Forbidden và các lần xác thực thất bại mã 401. Đây là công cụ đắc lực cho quản trị viên phát hiện nhanh các dấu hiệu tấn công dò quét lỗ hổng.

4.8 Truy vấn LogQL 3: Tìm sự kiện đăng nhập thất bại

Truy vấn nhắm trực tiếp vào stream nhật ký của ứng dụng, giải mã JSON và trích lọc chính xác sự kiện đăng nhập sai:

```
{job="app"} | json | event="login_failed"
```



Hình 4.5: Truy vấn LogQL 3 - Tìm sự kiện đăng nhập thất bại của ứng dụng

Kết quả trả về ghi nhận chi tiết thời điểm người dùng cố gắng đăng nhập sai tài khoản admin, mã phản hồi 401, địa chỉ IP nguồn và đường dẫn /login. Điểm đặc biệt quan trọng về mặt an toàn thông tin là dòng log tuyệt đối không lưu trữ mật khẩu nhập vào của người dùng, đảm bảo tuân thủ triệt để nguyên tắc bảo mật thông tin nhạy cảm.

Mã Truy Vấn	Biểu thức LogQL	Sự kiện kiểm thử kích hoạt	Dữ liệu thực tế ghi nhận
Query 1	{job="nginx"}	Duyệt web thông thường, mở dashboard, danh mục dự án	10+ dòng access log JSON đầy đủ method, path, status 200
Query 2	{job="nginx"} | json | status >= 400	Truy cập URL lạ /not-found-demo, CSRF denied	8+ dòng log lọc chính xác status 404 và 403
Query 3	{job="app"} | json | event="login_failed"	Cố tình nhập sai mật khẩu admin trên giao diện	Dòng audit log chứa event=login_failed, status=401, ẩn password

Bảng 4.2: Ba truy vấn LogQL kiểm chứng hệ thống logging tập trung

PHẦN 5: KIỂM THỬ VẬN HÀNH VÀ ĐỐI CHIẾU

5.1 Kết quả kiểm thử chức năng và tích hợp hệ thống

Nhóm triển khai đã xây dựng kịch bản kiểm thử tự động hóa toàn diện bằng script scripts/smoke_test.py, bao gồm 15 ca kiểm tra độc lập trên môi trường container thực tế. Kết quả kiểm thử đạt tỷ lệ thành công tuyệt đối 15/15 ca kiểm thử (100% PASS):

STT	Ca kiểm thử tự động	Kết quả thực tế ghi nhận	Đánh giá
01	Chuyển hướng HTTP sang HTTPS	Gửi request cổng 8080 nhận HTTP 301 chuyển hướng sang https://127.0.0.1:8443/	PASS
02	Health check & Kết nối Database	Endpoint /health phản hồi HTTP 200: {status: UP, db: CONNECTED, user: pm_app}	PASS
03	Kiểm tra Security Headers	X-Content-Type-Options: nosniff; X-Frame-Options: DENY; HSTS: active	PASS
04	Kiểm soát truy cập (Access Control)	Yêu cầu /projects khi chưa login lập tức bị redirect về /login	PASS
05	Tạo và kiểm tra CSRF Token	Form đăng nhập tự động nhúng mã CSRF token ngẫu nhiên độ dài an toàn	PASS
06	Sự kiện Đăng nhập thất bại	Đăng nhập sai nhận HTTP 401; log ứng dụng sinh event login_failed	PASS
07	Xác thực Đăng nhập thành công	Đăng nhập đúng tài khoản admin nhận phiên làm việc, chuyển hướng vào Dashboard	PASS
08	Cờ an toàn của Session Cookie	pm_session_id được thiết lập với đầy đủ cờ HttpOnly, SameSite=Lax, Secure=true	PASS
09	Thao tác CRUD: Thêm dự án mới	Gửi POST tạo dự án PRJ-TEST-332, cơ sở dữ liệu tạo bản ghi thành công ID: 7	PASS
10	Thao tác CRUD: Đọc chi tiết dự án	Truy cập /projects/7 xác nhận nội dung vừa tạo hiển thị chính xác trên giao diện	PASS
11	Thao tác CRUD: Xóa dự án	Gửi yêu cầu xóa dự án ID: 7, cơ sở dữ liệu xóa thành công bản ghi thử nghiệm	PASS
12	Cơ chế bảo vệ chống CSRF	Gửi POST không mang theo CSRF token bị hệ thống chặn đứng với mã HTTP 403 Forbidden	PASS
13	Xử lý lỗi HTTP 404	Truy cập đường dẫn lạ trả về trang 404 thiết kế chuẩn và ghi log cảnh báo	PASS
14	Prometheus Targets Active	API /api/v1/targets xác nhận 4/4 targets (cadvisor, nginx, postgres, prometheus) đều UP	PASS
15	Grafana Server & Datasource	Grafana phản hồi HTTP 200, nạp thành công 2 datasource Prometheus và Loki	PASS

Bảng 5.1: Kết quả kiểm thử chức năng và tích hợp hệ thống (15/15 ca kiểm thử PASS)

5.2 Quy trình sao lưu và phục hồi dữ liệu độc lập

Công tác bảo toàn dữ liệu được thực hiện tự động hóa qua script scripts/backup.py. Quá trình sao lưu sử dụng tiện ích chuẩn pg_dump của PostgreSQL với tùy chọn --clean --if-exists, tạo ra bản sao lưu pm_db_backup_20261021_234722.sql dung lượng 33,671 bytes nằm an toàn trong thư mục backups ngoài Git.

Kiểm chứng phục hồi dữ liệu độc lập: Nhằm đảm bảo an toàn tuyệt đối và không gây ảnh hưởng tới cơ sở dữ liệu đang hoạt động, script scripts/restore_test.py tạo ra một cơ sở dữ liệu độc lập hoàn toàn mang tên pm_restore_test, sau đó thực hiện nạp toàn bộ bản sao lưu vào database thử nghiệm này. Kết quả đối chiếu số lượng bản ghi giữa database gốc và database phục hồi đạt tỷ lệ khớp chính xác 100%:

Tên Bảng Dữ Liệu	Số bản ghi nguồn (Source)	Số bản ghi phục hồi (Restored)	Kết quả đối chiếu
users	3	3	KHỚP 100%
members	5	5	KHỚP 100%
projects	4	4	KHỚP 100%
tasks	9	9	KHỚP 100%
project_members	10	10	KHỚP 100%
audit_logs	11	11	KHỚP 100%

Bảng 5.2: Kết quả backup và phục hồi thử nghiệm độc lập trên cơ sở dữ liệu riêng

Sau khi hoàn tất kiểm tra tính toàn vẹn của dữ liệu, database thử nghiệm được dọn dẹp an toàn. Thử nghiệm khởi động lại (restart) container pm-app và pm-postgres chứng minh toàn bộ dữ liệu dự án, công việc và thành viên vẫn được bảo toàn nguyên vẹn trên named volume pm_postgres_data.

5.3 Các sự cố kỹ thuật gặp phải và giải pháp khắc phục

Trong suốt quá trình thiết lập và triển khai hệ thống thực tế, nhóm đã ghi nhận và xử lý bài bản 4 sự cố kỹ thuật điển hình:

Sự cố kỹ thuật phát sinh	Nguyên nhân gốc rễ	Giải pháp khắc phục triệt để
1. Cạn kiệt ổ đĩa C: (0 MB) khi kéo Docker image	Docker Desktop WSL2 tải nhiều layer image nặng cùng lúc trong khi ổ C: của máy chủ còn ít dung lượng trống	Phát hiện và di chuyển an toàn 2 file cài đặt lớn (~9.8 GB) sang phân vùng E:\ProjectManagement\temp_downloads (thêm vào .gitignore), giải phóng 9.8 GB cho ổ C:.
2. pgAdmin 8.14	Phiên bản pgAdmin	Cập nhật biến môi trường

từ chối email định dạng .local	8.14 áp dụng bộ kiểm tra Regex email RFC nghiêm ngặt, từ chối các domain nội bộ như .local	PGADMIN_DEFAULT_EMAIL=dtc245220019@ictu.edu.vn chuẩn định dạng giáo dục hợp lệ.
3. Xung đột cổng máy chủ (Port 3000/3001)	Máy trạm đang chạy các container của dự án khác trên cổng 3000 và 3001	Ánh xạ cổng dịch vụ Grafana sang cổng 3002 (127.0.0.1:3002), giữ nguyên toàn bộ các container và dịch vụ khác trên máy.
4. Nginx Alpine symlink log ra /dev/stdout	Image nginx:alpine mặc định liên kết access.log/error.log vào luồng console, khiến Promtail không thể đọc được file trên volume	Cấu hình container init-logs và lệnh container un-link các symlink, tạo file log vật lý thực tế trong named volume pm_nginx_logs để Promtail tail dữ liệu.

Bảng 5.3: Các sự cố kỹ thuật gặp phải trong quá trình triển khai và cách khắc phục

5.4 Bảng đối chiếu tiêu chí đánh giá và kịch bản demo

Hệ thống đáp ứng toàn diện và vượt mức các yêu cầu kỹ thuật do giảng viên đề ra trong học phần:

Tiêu chí đánh giá của đề bài	Minh chứng kỹ thuật tương ứng trong đồ án	Mức độ hoàn thành
1. Quản lý mã nguồn trên GitHub	Repository công khai, tài liệu README chi tiết, 3 commit mốc theo chuẩn Git Flow	100% Hoàn thành
2. Ứng dụng Quản lý Dự án & DB	Đầy đủ nghiệp vụ Projects, Tasks, Members, Audit Logs; PostgreSQL 16 + pgAdmin 4	100% Hoàn thành
3. Docker Compose & Nginx	Compose mô-đun, Nginx SSL termination RSA 3072 bit, chuyển hướng HTTP->HTTPS, Security Headers	100% Hoàn thành
4. Giám sát Prometheus & Grafana	Prometheus scrape 4 targets UP (100%), Grafana auto-provisioning dashboard chuyên sâu	100% Hoàn thành
5. Nhật ký Loki & Promtail	Log JSON có cấu trúc, 3 truy vấn LogQL thực thi thành công minh chứng truy vết sự cố	100% Hoàn thành
6. Gia cố an toàn hệ thống (Hardening)	Non-root UID 1000, read-only rootfs, cap_drop ALL, internal DB network, least privilege	100% Hoàn thành
7. Kiểm thử & Sao lưu phục hồi	15/15 ca smoke test PASS, sao lưu pg_dump và phục hồi thử nghiệm độc lập khớp 100%	100% Hoàn thành
8. Báo cáo Word chuẩn mực	Báo cáo >10 trang, bìa đúng mẫu ICTU, số liệu và hình ảnh chụp thực tế từ hệ thống đang chạy	100% Hoàn thành

Bảng 5.4: Bảng đối chiếu tiêu chí chấm điểm và kết quả đạt được

Kịch bản Demo hệ thống từng bước tại buổi bảo vệ

Bước	Nội dung trình diễn Demo	Thao tác thực hiện	Kết quả mong đợi
1	Kiểm tra mã nguồn & Git	Mở trình duyệt xem GitHub repo và gõ lệnh: git log --oneline	Hiển thị 3 commit mốc kỹ thuật, README và cây thư mục chuẩn.
2	Khởi chạy hệ thống	Thực thi: docker compose -f compose.yaml -f compose.monitoring.yaml -f compose.logging.yaml ps	11 containers ở trạng thái running (Up), init-logs ở trạng thái Exited (0).
3	Truy cập Web & Nginx HTTPS	Mở trình duyệt truy cập http://localhost:8080	Tự động chuyển hướng sang https://localhost:8443, hiển thị giao diện đăng nhập.
4	Thao tác nghiệp vụ ứng dụng	Đăng nhập tài khoản admin, thêm dự án mới, cập nhật trạng thái task, gán thành viên	Dữ liệu được lưu trữ tức thời, Dashboard cập nhật tỷ lệ hoàn thành, Audit Logs ghi nhận.
5	Đối chiếu trên pgAdmin 4	Truy cập http://localhost:8081, xem bảng projects và tasks trong database pm_db	Dữ liệu vừa thêm xuất hiện đầy đủ trong PostgreSQL.
6	Trình diễn Prometheus	Truy cập http://localhost:9090/targets	Hiển thị 4 targets (prometheus, cadvisor, nginx, postgres) đều màu xanh UP.
7	Trình diễn Grafana Dashboard	Truy cập http://localhost:3002, mở Dashboard Project Management	Hiển thị trực quan đồ thị CPU, RAM, Network và số kết nối PostgreSQL theo thời gian thực.
8	Thực thi 3 truy vấn LogQL	Vào Grafana Explore (Loki), chạy lần lượt 3 câu LogQL đã cấu hình	Trả về dòng log thật: access log Nginx, lọc lỗi >= 400 và sự kiện login_failed.
9	Minh chứng Hardening	Chạy scripts/verify_hardening.py kiểm tra UID, read-only rootfs, cap_drop, internal net	Hiển thị kết quả xác minh bảo mật đạt chuẩn.
10	Minh chứng Backup/Restore	Chạy scripts/backup.py và scripts/restore_test.py	Tạo file backup SQL và phục hồi thử nghiệm đối

			chiếu khớp 100% bản ghi.

Bảng 5.5: Kịch bản demo hệ thống từng bước

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án đã hoàn thành xuất sắc 100% các mục tiêu đề ra cho Đề tài số 14: "Hệ thống Quản lý Dự án (Project Management)" theo đúng tiêu chuẩn yêu cầu của học phần Triển khai và Quản trị Hệ thống Phần mềm. Hệ thống không chỉ đáp ứng hoàn chỉnh các chức năng nghiệp vụ phần mềm mà còn thiết lập thành công một hạ tầng vận hành container hóa hiện đại, bảo mật và có khả năng quan sát toàn diện.

Những kết quả nổi bật đã đạt được gồm:

1. Đóng gói và điều phối chuẩn mực: 100% các thành phần (App, Database, Nginx, Prometheus, Grafana, Loki, Promtail, cAdvisor, Exporters) được đóng gói độc lập và quản lý thống nhất bằng Docker Compose với healthcheck và named volumes bền vững.

2. An toàn thông tin đa tầng: Thiết lập HTTPS tự ký với chứng chỉ RSA 3072 bit, đầy đủ HTTP security headers, chạy non-root user (UID 1000), hệ thống tệp chỉ đọc (read-only rootfs), loại bỏ toàn bộ Linux Capabilities nguy hiểm, mạng database internal cô lập hoàn toàn và phân quyền tài khoản cơ sở dữ liệu theo nguyên tắc đặc quyền tối thiểu.

3. Khả năng quan sát (Observability) vượt trội: Thu thập tự động 4 luồng metrics với 100% targets UP; trực quan hóa trực quan trên Grafana; quản lý tập trung toàn bộ nhật ký ứng dụng và web server bằng Loki/Promtail với 3 truy vấn LogQL chuyên sâu.

4. Kiểm thử và sao lưu tin cậy: Toàn bộ 15 ca kiểm thử tự động đạt kết quả PASS; quy trình sao lưu và phục hồi thử nghiệm độc lập chứng minh tính toàn vẹn 100% của dữ liệu mà không làm gián đoạn hệ thống chính.

Các mặt còn hạn chế của đồ án:

• Hệ thống hiện được triển khai trên môi trường đơn máy chủ (Single-node Docker Host), chưa mở rộng theo quy mô cụm phân tán (Cluster).

• Chứng chỉ số TLS hiện tại là chứng chỉ tự ký (Self-signed) phục vụ môi trường kiểm thử nội bộ localhost, chưa phải là chứng chỉ được cấp bởi CA công cộng tin cậy (như Let's Encrypt).

• Hệ thống giám sát mới dừng lại ở việc hiển thị trực quan, chưa tích hợp kênh gửi cảnh báo tức thời (Alertmanager thông báo qua Telegram hoặc Email khi có sự cố).

Định hướng nghiên cứu và phát triển tiếp theo:

• Nâng cấp hệ thống lên nền tảng điều phối Kubernetes (K8s) nhằm hỗ trợ tự động mở rộng (Auto-scaling) và tăng cường tính sẵn sàng cao (High Availability).

• Tích hợp cơ chế tự động cấp phát và gia hạn chứng chỉ SSL miễn phí thông qua Certbot / Let's Encrypt khi triển khai trên tên miền Internet thực tế.

• Bổ sung mô-đun Alertmanager kết nối Webhook gửi thông báo tự động tới kênh Telegram của đội ngũ vận hành khi dịch vụ bị DOWN hoặc tỷ lệ lỗi HTTP tăng đột biến.

• Xây dựng đường ống tích hợp và triển khai liên tục (CI/CD Pipeline) với GitHub Actions để tự động hóa toàn bộ quy trình kiểm thử, build Docker image và deploy lên máy chủ.

TÀI LIỆU THAM KHẢO

[1] Docker Inc., “Docker Compose File Reference & Networking Documentation”, 2026. https://docs.docker.com/compose/

[2] Nginx Software Inc., “NGINX Reverse Proxy, SSL Termination and Security Controls”, 2026. https://nginx.org/en/docs/

[3] PostgreSQL Global Development Group, “PostgreSQL 16 Reference Manual: Security and Administration”, 2026. https://www.postgresql.org/docs/16/

[4] Prometheus Authors, “Prometheus Querying Basics, Metrics Types and Best Practices”, 2026. https://prometheus.io/docs/

[5] Grafana Labs, “Grafana Dashboards, Datasource Provisioning and Alerting Guide”, 2026. https://grafana.com/docs/grafana/latest/

[6] Grafana Labs, “Grafana Loki: LogQL Query Language & Architecture Guide”, 2026. https://grafana.com/docs/loki/latest/

[7] Open Web Application Security Project (OWASP), “OWASP Top 10 Web Application Security Risks”, 2025. https://owasp.org/www-project-top-ten/

[8] Center for Internet Security (CIS), “CIS Docker Benchmark v1.6.0: Hardening Container Environments”, 2024. https://www.cisecurity.org/benchmark/docker

[9] Express.js Foundation, “Production Best Practices: Performance and Reliability in Node.js”, 2026. https://expressjs.com/en/advanced/best-practice-performance.html

[10] Đinh Gia Bảo, “Repository mã nguồn và cấu hình Đề 14: ProjectManagement_TKQTHT”, GitHub, 2026. https://github.com/dtc245220019-create/ProjectManagement_TKQTHT